Names: Harshita Singh , Prerna Bahrani , Gargi Shintre , Abhishek Vishwakarma
Roll nos: 33, 41, 32, 35
Class : D20C
Subject: BlockChain

Experiment No: 7

AIM:

To study the Truffle development framework, establish a local Ethereum blockchain using Ganache, connect Ganache with MetaMask and Remix IDE, and compile, deploy, and interact with a Solidity smart contract based on the TrustCart mini-project.

THEORY

1. Blockchain

Blockchain is a distributed and append-only digital ledger that records transactions in blocks. Each block is linked to the previous block using cryptographic hashes, making unauthorized modification of recorded data difficult to conceal.

In the retail and e-commerce sector, blockchain can be used to maintain product records, track supply-chain movements, verify product information, and improve transparency between manufacturers, distributors, retailers, and customers.

2. Truffle Framework

Truffle is a blockchain development framework used to develop, compile, test, and deploy Ethereum smart contracts. It provides features such as contract compilation, migration scripts, automated testing, and interaction with blockchain networks.

The Truffle framework traditionally works with Ganache as a local blockchain. In this experiment, Remix IDE is used for writing, compiling, deploying, and interacting with the smart contract, while Ganache provides the local blockchain environment.

3. Ganache

Ganache is a personal Ethereum blockchain used for smart contract development and testing. It provides a local blockchain environment where developers can deploy contracts and execute transactions without using a public blockchain or real Ether.

Features of Ganache:

- Provides pre-funded test accounts.
- Allows developers to deploy and test smart contracts locally.
- Displays transaction history and block information.
- Provides transaction details such as sender address, transaction hash, gas usage, and block number.
- Enables rapid development and debugging of decentralized applications.

4. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that allows users to connect decentralized applications to blockchain networks. It manages blockchain accounts and allows users to approve transactions.

In this experiment, MetaMask is connected to the Ganache local network by configuring the RPC URL and chain ID and importing a Ganache test account. Remix uses MetaMask to send transactions to the local blockchain.

5. Remix IDE

Remix IDE is a browser-based development environment for writing, compiling, deploying, and interacting with Solidity smart contracts.

Its Solidity Compiler compiles source code into bytecode and an ABI. The bytecode is used for deployment, while the ABI describes the contract's callable functions and their parameters.

By selecting **Injected Provider – MetaMask** in the Deploy & Run Transactions panel, Remix can interact with the blockchain network selected in MetaMask.

6. Smart Contracts

A smart contract is a program deployed on a blockchain that executes predefined logic when its functions are called.

In this experiment, a `ProductPassport` smart contract is implemented for TrustCart. It registers a product with a unique product ID, product name, metadata string, manufacturer address, and registration timestamp. It also provides a function to retrieve and verify the registered product information.

The contract prevents duplicate product IDs and rejects invalid product registrations.

7. TrustCart Mini-Project Application

TrustCart aims to improve transparency in e-commerce by maintaining blockchain-based product records. A product passport can store a product's unique identifier and reference information about its origin or authenticity.

The prototype implemented in this experiment demonstrates the product-registration and verification components of the larger mini-project. It does not independently prove the authenticity of a physical product; the reliability of the original information depends on the parties providing it.

PROCEDURE

Step 1. Ganache Setup

- Launch **Ganache Desktop** and note the **RPC URL** ([http://127.0.0.1:7545]) and **Chain ID** (1337 / 5777).
- Copy the **Private Key** of a test account.

Step 2. MetaMask Setup

- Add a custom network: **Ganache Local** → http://127.0.0.1:7545 → Chain ID 1337 → Symbol ETH.
- Click **Import Account** and paste the Ganache private key.

Step 3. Remix Connection

- Open Remix IDE.
- Under **Deploy & Run**, set **Environment** to **Injected Provider / Browser Extension** and confirm the MetaMask popup.

Step 4. Compile & Attach Contract

- Create ProductPassport.sol and compile it with version 0.8.20 (EVM version set to **london**).
- In **Deploy & Run**, paste your existing contract address into **At Address** and click the button.

Step 5. Test & Verify

- **Register:** registerProduct(1001, "Premium Sneakers", "QmHash001") → confirm in MetaMask.
- **Verify:** verifyProduct(1001) → returns product details.
- **Revert Checks:** Re-register 1001 or check 9999 to test contract error rules.
- **Evidence:** Capture screenshots of Remix results, MetaMask activity, and Ganache transaction logs.

IMPLEMENTATION

ProductPassport.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract ProductPassport {

    struct Product {
        uint256 productId;
        string productName;
```

```
        string metadataHash;
        address manufacturer;
        uint256 registeredAt;
    }

    mapping(uint256 => Product)
private products;
    mapping(uint256 => bool) public
productExists;

    event ProductRegistered(
        uint256 indexed productId,
        string productName,
        string metadataHash,
        address indexed manufacturer,
        uint256 registeredAt
    );

    function registerProduct(
        uint256 _productId,
        string memory _productName,
        string memory _metadataHash
    ) public {
        require(_productId != 0,
"Invalid product ID");
        require(

!productExists[_productId],
            "Product already
registered"
        );
        require(

bytes(_productName).length > 0,
            "Product name required"
        );
        require(

bytes(_metadataHash).length > 0,
            "Metadata hash required"
        );

        products[_productId] =
Product({
            productId: _productId,
            productName:
_productName,
            metadataHash:
_metadataHash,
            manufacturer: msg.sender,
            registeredAt:
block.timestamp
        });

        productExists[_productId] =
true;

        emit ProductRegistered(
            _productId,
            _productName,
            _metadataHash,
            msg.sender,
            block.timestamp
        );
    }

    function verifyProduct(uint256
_productId)
        public
        view
        returns (
            bool exists,
            string memory
productName,
            string memory
metadataHash,
            address manufacturer,
            uint256 registeredAt
        )
    {
        require(

productExists[_productId],
            "Product not found"
        );

        Product memory p =
products[_productId];

        return (
            true,
            p.productName,
            p.metadataHash,
            p.manufacturer,
            p.registeredAt
        );
    }
}
```

Explanation of the Program

1. Product structure
The `Product` structure stores the product ID, name, metadata string, manufacturer's blockchain address, and registration timestamp.

2. Product mapping
The `products` mapping associates each product ID with its corresponding product record. The `productExists` mapping indicates whether a product ID has already been registered.

3. Product registration
The `registerProduct()` function registers a product on the blockchain. It checks that the ID is non-zero, the ID is not already registered, and the name and metadata string are not empty.

4. Manufacturer identification
The `msg.sender` variable records the blockchain address that submitted the registration transaction.

5. Timestamp
The `block.timestamp` variable records the timestamp associated with the block in which the registration is executed.

6. Event generation
The `ProductRegistered` event records important details of a successful registration. Events can be inspected through transaction logs and used by decentralized applications to track contract activity.

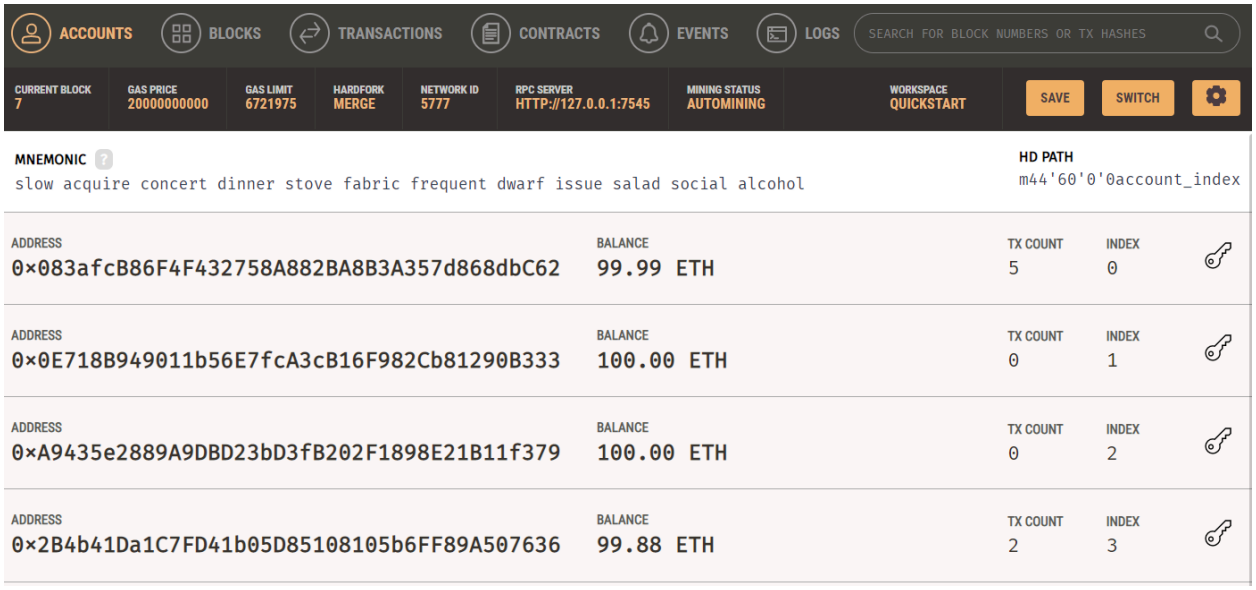
7. Product verification
The `verifyProduct()` function retrieves the stored product information. It uses the `view` modifier because it reads contract state without modifying it.

8. Validation
The contract prevents duplicate IDs and rejects attempts to verify an unregistered product.
Note: The `metadataHash` field in this experiment is a string supplied by the user. The contract does not calculate or validate a cryptographic hash, and the example string is not a real product-metadata hash.

OUTPUT

1. Ganache Environment

Ganache displays the generated accounts, their balances, and the transaction history of the local blockchain.



2. Smart Contract Compilation and Deployment

Remix successfully compiles `ProductPassport.sol` using a compatible Solidity compiler.

```
PS C:\Users\Harshita\Desktop\college\BLOCKCHAIN\TrustCart-Experiment7> truffle compile

  Compiling your contracts...
  ===========================
  ✓ Fetching solc version list from solc-bin. Attempt #1
  ✓ Downloading compiler. Attempt #1.
  > Compiling .\contracts\ProductPassport.sol
  > Artifacts written to C:\Users\Harshita\Desktop\college\BLOCKCHAIN\TrustCart-Experiment7\build\contracts
  > Compiled successfully using:
     - solc: 0.8.21+commit.d9974bed.Emscripten.clang
```

```
PS C:\Users\Harshita\Desktop\college\BLOCKCHAIN\TrustCart-Experiment7> truffle migrate --network development
Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)


2_deploy_product_passport.js
============================

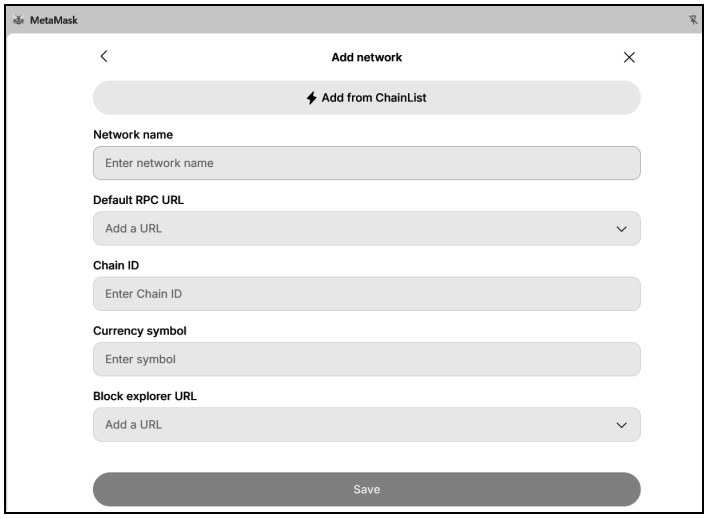
   Deploying 'ProductPassport'
   ---------------------------
   > transaction hash:    0x4e88a0f78f383a105f68c9c58692f65543e4a1e5492e457e292773dc519ebadd
   > Blocks: 0            Seconds: 0
   > contract address:    0xD7E3d1dc33E26dBe507ae37E4308f25D3C33423C
   > block number:        1
   > block timestamp:     1792578596
   > account:             0x083afcB86F4F432758A882BA8B3A357d868dbC62
   > balance:             99.997066855
   > gas used:            869080 (0xd42d8)
   > gas price:           3.375 gwei
   > value sent:          0 ETH
   > total cost:          0.002933145 ETH

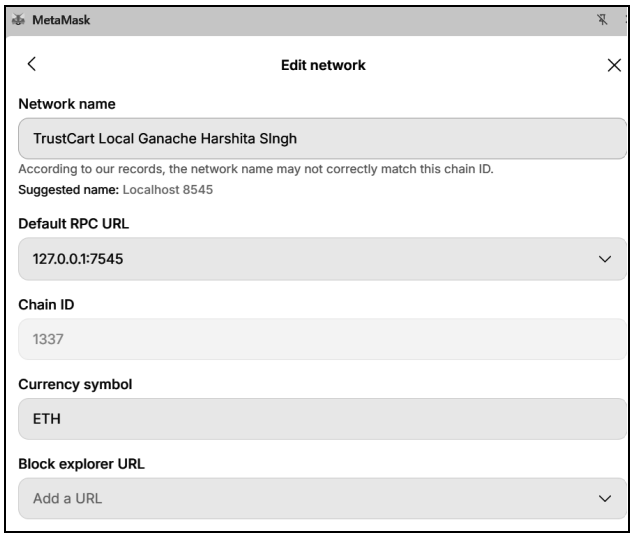
   > Saving artifacts
   -------------------------------------
   > Total cost:         0.002933145 ETH
```

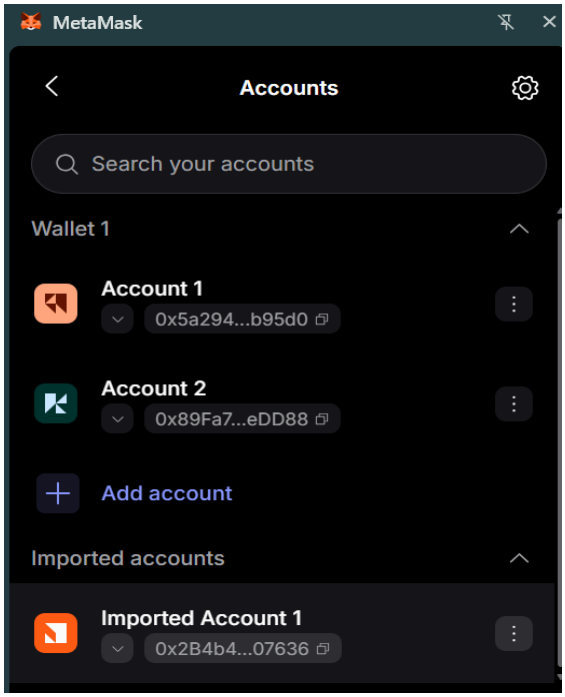
3. Smart Contract Deployment

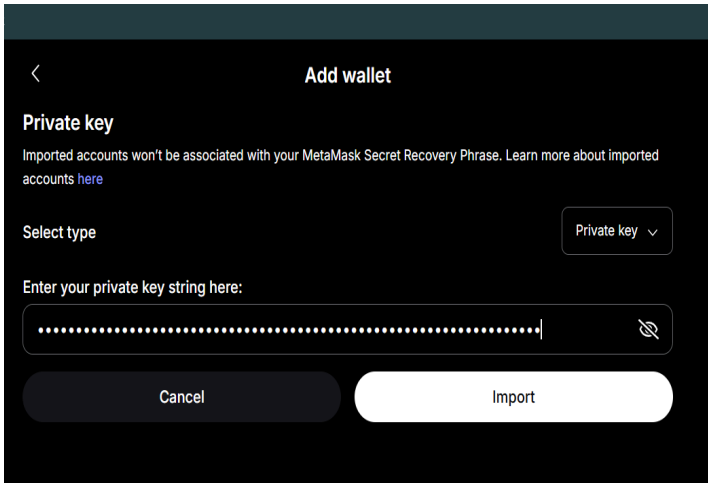
The contract is deployed to Ganache through Remix and MetaMask. Remix displays the deployed contract and its address.

4. Connect Ganache With Metamask

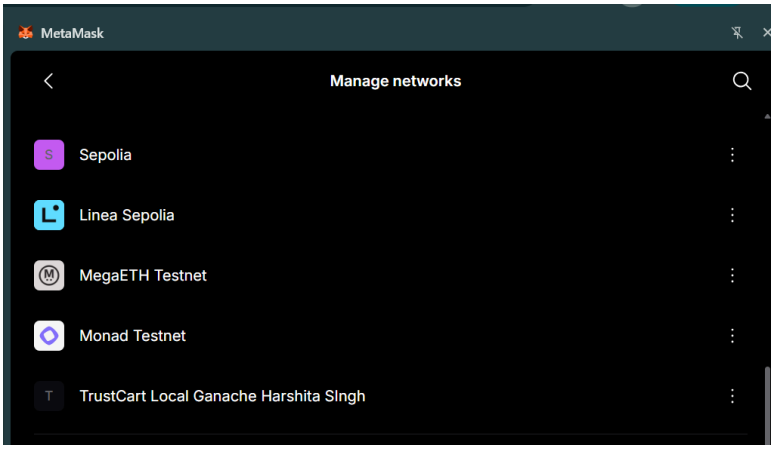




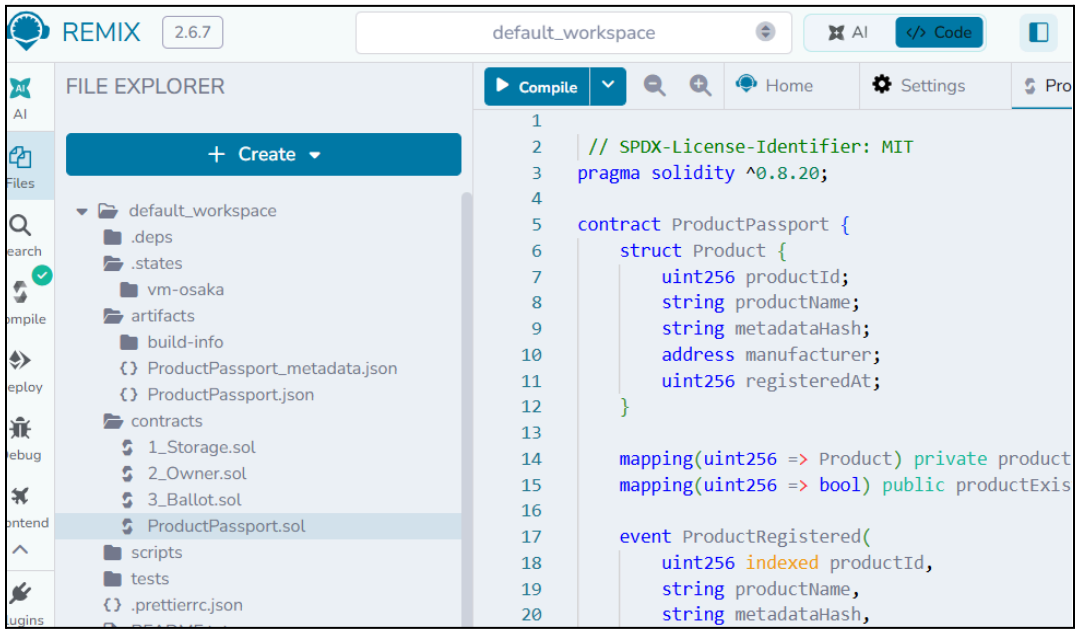


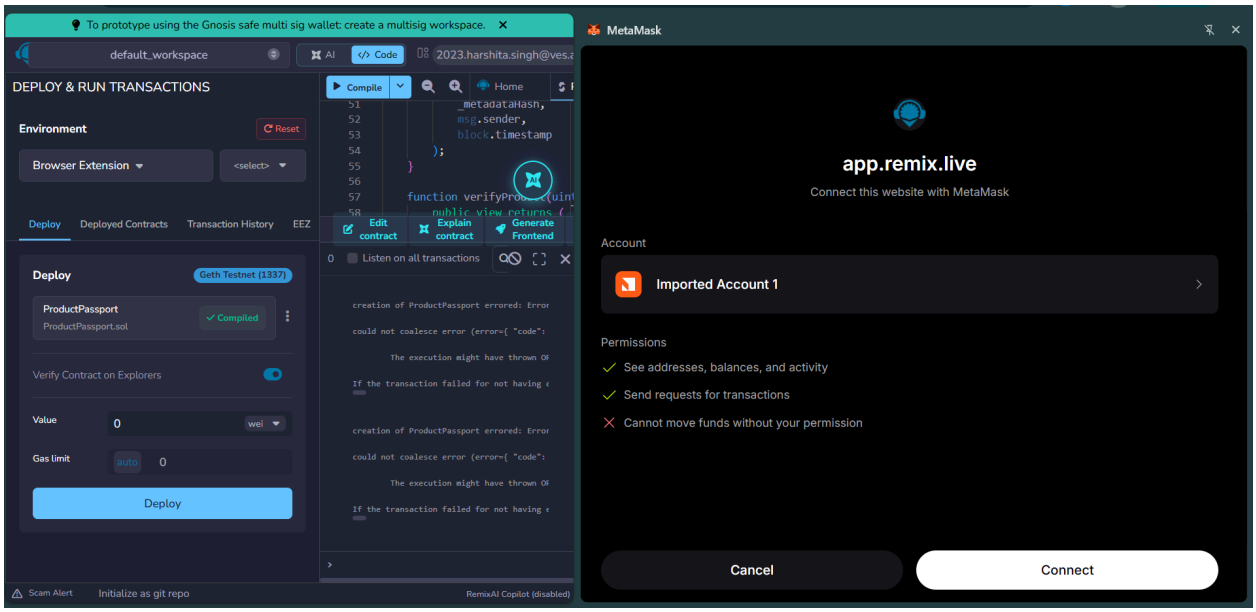


You can see the created network:



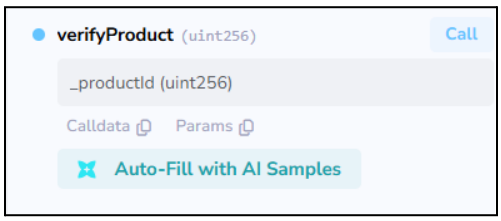
5. Product Existence Verification

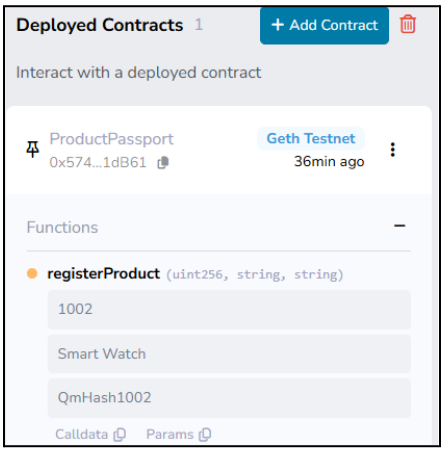


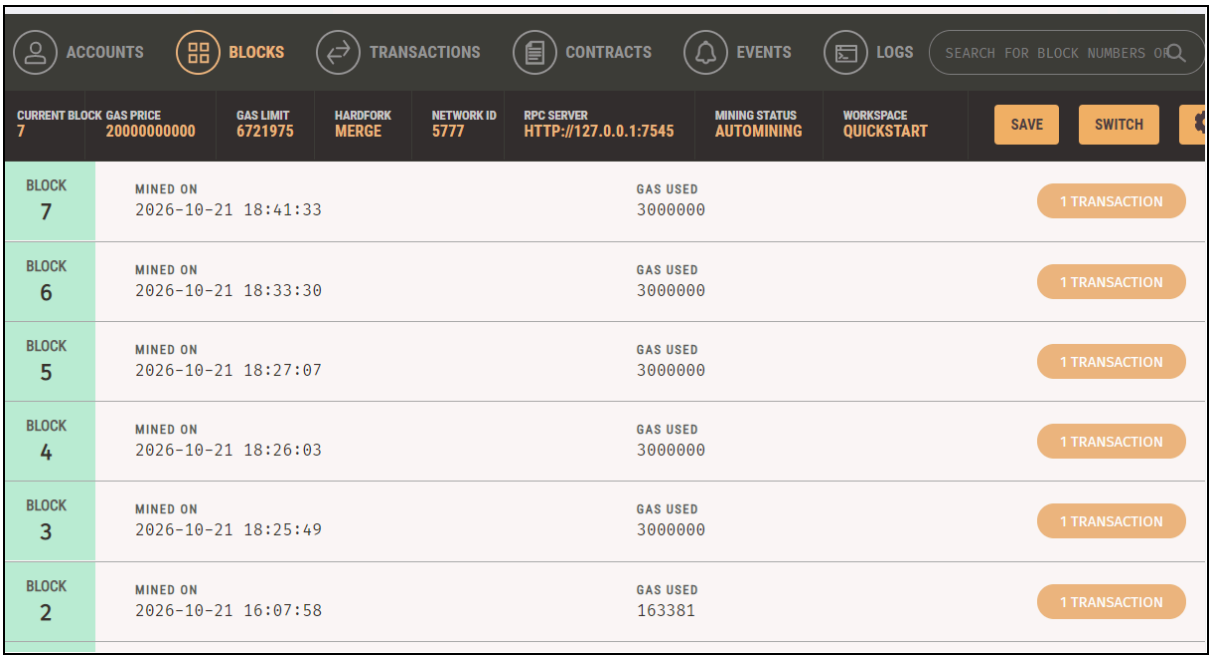


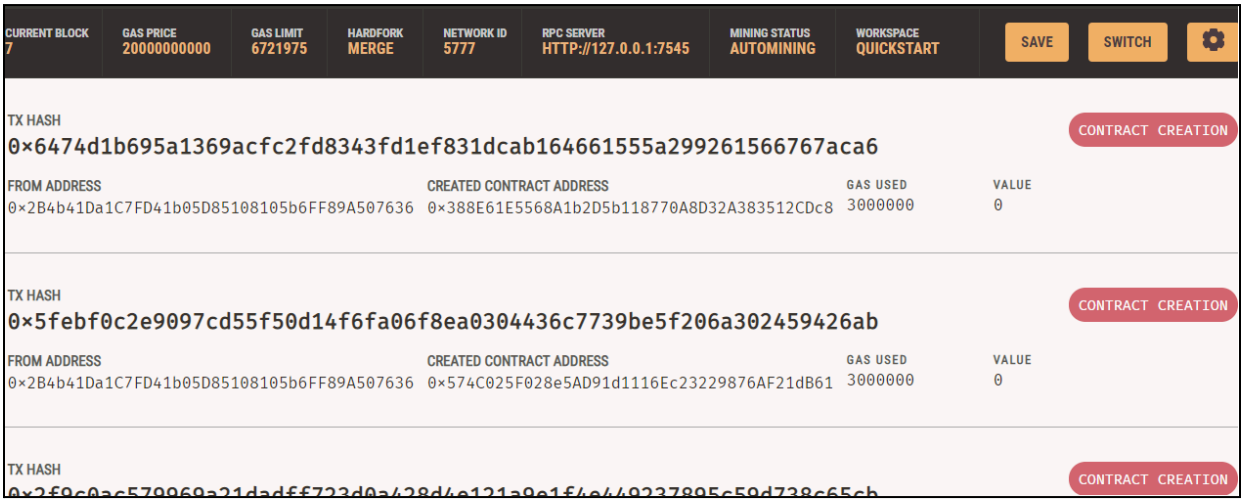
6. Smart Contract Deployed:











CONCLUSION

Ganache provides a convenient environment for simulating a local Ethereum blockchain, while MetaMask enables account management and transaction approval, and Remix IDE provides tools for compiling, deploying, and interacting with Solidity smart contracts.

The `ProductPassport` smart contract demonstrates how blockchain can support product registration and verification in the TrustCart e-commerce mini-project. Product records can be associated with unique identifiers and manufacturer addresses, while validation rules prevent duplicate registrations. The experiment provides a foundation for extending the mini-project with supply-chain tracking, retailer verification, product recalls, and warranty management